
ODI Demo Builder — Session Guide

~3 hours · No skill install required — paste prompts directly into Claude Code

How this works. Each phase has a prompt in a red-bordered box — paste it into Claude Code. Share a screen on the Meet so results are visible to everyone. When something breaks, tell Claude Code exactly what happened and ask it to fix the issue. **Keep the same Claude Code session open for all four phases.** Do not close or restart it between phases — the later prompts rely on context from earlier ones.

Before you start

5 minutes

Open a terminal in the directory where you want the demo project created. Run `claude` from there. Every file Claude Code generates lands relative to that directory. Keep that terminal open for the full session.

Sign in to 1Password first. Running `op signin` directly will print an error telling you to use `eval` instead — that is expected. The correct command is:

```
eval $(op signin)
```

Enter your 1Password credentials when prompted. Then run the full verification block — every command should return output, not an error.

```
claude whoami
op whoami
terraform version
dbt --version
node --version
npm --version
jq --version
aws sts get-caller-identity --query Account --output text
echo "Fivetran key: ${FIVETRAN_API_KEY:0:8}..."
echo "Fivetran secret: ${FIVETRAN_API_SECRET:0:8}..."
```

If anything fails, paste the error into Claude Code: "This command failed — how do I fix it?"

Share your screen on the Meet now. Keep the same person sharing for the full session. When your outputs differ, the shared screen is the reference.

Agree on these three things before pasting any prompt. Write them down or paste into the Meet chat.

1. **Industry vertical and fictional company name.** The name must not be a real company. Examples that work: Aldermoor Systems, Crestfield Analytics, Vantage Meridian. Claude Code will validate your choice.
2. **Buyer persona** — CDO, CIO, or Head of Data Engineering. This shapes the frontend copy.

3. **Your 1Password vault name and item name** for the SQL Server credentials. Claude Code will use these to construct `op://vault/item/field` URIs — you need both the vault name and the item name, not just the item name.

When something breaks, do this every time:

1. Do not try to fix the error by editing files or rerunning commands yourself first.
2. Copy the full error output — the complete text, not just the last line.
3. Paste it into Claude Code and say: **"This failed. Here is the error. What is the fix?"**
4. Claude Code will diagnose it and give you the exact steps.

If Claude Code cannot resolve the error within 15 minutes, the issue is likely outside your control (firewall, missing IAM permissions, disabled CDC on the SQL Server). Pause and escalate to whoever administers that system before continuing.

Phase 1 — Gather

target: 15 minutes

PHASE 1 — GATHER

Claude Code collects the demo context, pulls SQL Server credentials from 1Password, tests connectivity, lists tables, and gets your update-method decision.

PASTE INTO CLAUDE CODE — BOTH OF YOU

You are facilitating a hands-on Fivetran ODI demo build session. Your job is to guide the build of a production-ready ODI demo from scratch — SQL Server as the source, Fivetran as the ingest layer, Apache Iceberg on S3 as the lake, dbt for bronze/silver/gold transforms, and a React frontend as the demo surface.

Narrate your reasoning at every decision point.

Start by collecting five inputs:

1. Industry vertical and fictional company name — validate that the name does not collide with a real entity before proceeding. Check your training knowledge for any registered business, publicly traded company, or well-known brand with that exact name. Reject it if you find any match.
2. Buyer persona: CDO, CIO, or Head of Data Engineering
3. The 1Password vault name AND item name for SQL Server credentials — ask for both separately. Construct four `op read` URIs in the format `op://vault-name/item-name/field-name` for host, port, username, and password (standard 1Password field names are "server", "port", "username", "password"). Run each as: `export FIELD=$(op read "op://vault-name/item-name/field-name")`. Never prompt for credentials in plaintext.
4. AWS region (default us-east-1) and S3 bucket name prefix
5. Target database and table selection — list these after connectivity is confirmed

Ask for all five now. Do not start any infrastructure work until you have them.

Answer Claude Code's questions. For 1Password, give both your vault name and item name when asked — it will construct the `op://` URIs itself.

When Claude Code asks about the update method, it will present three options: XACT (transaction-based log reading, no SQL Server Agent required — recommended), LOG_BASED (uses SQL Server replication, more setup required), or RELOAD (full table scan, no special permissions). If you do not administer the SQL Server and cannot confirm CDC is enabled, choose RELOAD — you can change it in the Fivetran dashboard later. To verify CDC readiness before choosing XACT, run on the SQL Server: `SELECT name, is_cdc_enabled FROM sys.databases WHERE name = 'your_db' .`

Phase 1 done when: connectivity confirmed, tables selected, update method decided.

Keep this Claude Code session open. Do not close or restart it. Phase 2, 3, and 4 all reference values Claude Code collected here. If you restart, paste the Phase 1 prompt again and re-answer the questions before continuing.

Phase 2 — Provision

target: 60 minutes

PHASE 2 — PROVISION

Before pasting the Phase 2 prompt, go to fivetran.com and start creating a new MDLS destination. Work through the wizard until you see both values: the **Fivetran AWS account ID** and the **External ID**. The External ID appears mid-wizard after you name the destination — not on the first screen. Copy both values — you need them for Terraform.

PASTE INTO CLAUDE CODE — BOTH OF YOU

This continues the Phase 1 conversation. Use the demo slug, company name, SQL Server credentials, update method, AWS region, and S3 bucket prefix already established above.

Run Phase 2: Provision. In order:

1. Run `echo $HOME` to get the home directory. Use the demo slug from Phase 1 to construct the full project path as `$HOME/Documents/GitHub/{demo_slug}-odi-demo/infra/` — resolve both values from Phase 1 context before creating the directory. Initialize a git repo there.
2. Write `main.tf` with: S3 bucket (versioning enabled, AES256 encryption, public access blocked, 30-day lifecycle to `STANDARD_IA` scoped to the data/ prefix only), three Glue databases (`{demo_slug}_bronze`, `{demo_slug}_silver`, `{demo_slug}_gold`), IAM role for Fivetran MDLS with trust policy using `fivetran_aws_account_id` and `fivetran_external_id` as variables, Athena workgroup, and Terraform outputs for `bucket_name`, `fivetran_role_arn`, `bronze_db`, `silver_db`, `gold_db`.
3. Write `terraform.tfvars` with placeholders for `fivetran_aws_account_id` and `fivetran_external_id`. Pause and ask me to fill in the two real values from the Fivetran MDLS setup wizard before running anything.
4. After I confirm the real values are in `tfvars`: run `terraform init`, then `terraform plan`. Print the plan and wait for my explicit approval before applying.
5. Run `terraform apply` after I approve. Run `terraform output -json` and save the full result to `infra/outputs.json`. Print each output value on a labeled line.
- 5a. Create the Fivetran group: `POST https://api.fivetran.com/v1/groups` with body `{"name": "{demo_slug}"}` using `FIVETRAN_API_KEY` and `FIVETRAN_API_SECRET` from the environment. Capture the returned `id` field. Print it as: `GROUP_ID`:

6. Create the Fivetran MDLS destination: POST `https://api.fivetran.com/v1/destinations` with body `{"group_id": GROUP_ID, "service": "managed_datalake", "config": {"bucket": bucket_name, "region": aws_region, "role_arn": fivetran_role_arn, "external_id": fivetran_external_id}}` – substitute all values from Terraform outputs and Phase 1.

7. Create the SQL Server connector: POST `https://api.fivetran.com/v1/connectors` with the 1Password credentials and the update method from Phase 1. Use the GROUP_ID from step 5a as the group_id. Construct the deep link as `https://fivetran.com/dashboard/connectors/{connector_id}` and print both on labeled lines: CONNECTOR_ID: and DEEP_LINK: . Store both in `infra/outputs.json`.

8. Before surfacing the Hybrid Deployment aside: give a 3-sentence explanation of Fivetran Hybrid Deployment – how it keeps customer data inside the customer network, what metadata the Fivetran cloud plane receives, and why the architecture built today works identically with a Hybrid Agent in front.

9. Test the connector: POST `https://api.fivetran.com/v1/connectors/{connector_id}/test`. Confirm `setup_state` equals "connected" (lowercase) before reporting Phase 2 complete. If `setup_state` is not "connected", print the full response and stop.

Show the Terraform plan before applying. Do not create the connector until the destination is confirmed.

Claude Code pauses twice: once for the Fivetran tfvars values, and once for the Terraform plan. Review the plan — you should see an S3 bucket, three Glue databases, an IAM role, and an Athena workgroup. Tell Claude Code "approved, apply it" when ready.

Phase 2 done when: Terraform applied, outputs.json written, MDLS destination created, SQL Server connector status equals "connected".

If the connector test does not return "connected": Paste the full JSON response to Claude Code and ask it to diagnose. It will tell you exactly what is wrong and how to fix it.

Sync gate — both confirm before proceeding. Confirm in the Meet chat that your connector shows "connected" before either of you pastes the Phase 3 prompt. If your partner is still in Phase 2, wait.

Phase 3 — Build

target: 60 minutes

PHASE 3 — BUILD

PASTE INTO CLAUDE CODE — BOTH OF YOU

This continues the Phase 1 and Phase 2 conversation. Use the demo slug, company name, connector ID, S3 bucket name, and Glue database names already established above. Read `infra/outputs.json` if you need to recover any Terraform output values.

Run Phase 3: Build.

In order:

1. Scaffold the dbt project in the project directory created in Phase 2 – resolve it as `$HOME/Documents/GitHub/{demo_slug}-odi-demo/dbt/` using the demo slug from Phase 1. Create

directories: models/bronze, models/silver, models/gold, seeds, macros, tests.

2. Write dbt_project.yml: project name from demo slug, Athena profile, bronze materialized as views, silver materialized as views, gold materialized as Iceberg tables using config(materialized="table", table_type="iceberg", s3_data_dir="s3://{bucket}/gold/", s3_tmp_table_dir="s3://{bucket}/tmp/") – table_type="iceberg" is required by dbt-athena, not optional.

3. Write packages.yml: [{package: "dbt-labs/dbt_utils", version: [">=1.0.0"]}, {package: "calogica/dbt_expectations", version: [">=0.10.0"]}]

4. Write profiles.yml using environment variables for all connection details – nothing hardcoded. Use the dbt-athena-community adapter (type: athena). Required fields: region_name, s3_staging_dir (s3://{bucket}/athena-results/), s3_data_dir (s3://{bucket}/dbt/), database: awsdatalog, schema: {demo_slug}_bronze, work_group: {demo_slug}.

5. Generate sources.yml in models/bronze/ for every table from Phase 1. Each table gets: loaded_at_field: _fivetran_synced, freshness warn_after 1 hour error_after 4 hours, a _fivetran_synced not_null test, and a _fivetran_deleted column documented.

6. For each source table: ask me for the primary key column, date columns, and metric/amount columns, then generate a staging model in models/silver/ using the standard source → renamed CTE pattern. Use try_cast instead of cast for numeric columns to handle dirty data gracefully.

7. Ask me to identify the main dimension entity and main fact grain, then generate at least one gold dim model and one gold fct model using Iceberg materialization.

8. Apply content rules to everything generated:

- No raw table or schema names in any string that renders in the UI – create a display-names constants file mapping real names to fictional labels; the UI imports from it
- Fictional org name used consistently in all copy
- pipeline.json must include connector ID and full deep link URL for every connector

9. Generate the React frontend shell using: npm create vite@latest frontend -- --template react-ts (do not use create-react-app – it prompts interactively). Then build: App.tsx with HashRouter, Landing page with org name and buyer-persona copy, Pipeline page with connector cards that include working Open-in-Fivetran links, Analysis page with a Recharts chart that shows axis labels, a chart title using the fictional org name, and a note that live data populates after gold models complete.

10. Validate the dbt project without running any queries against Athena: run dbt deps && dbt compile && dbt ls --select "path:models/silver path:models/gold". This confirms syntax and model structure without requiring source data. The full dbt build runs in Phase 4 after the first sync completes.

11. Report compile and ls results. If anything fails, paste the error and diagnose before suggesting a fix.

Ask me about primary keys and grain before writing any SQL. If you have more than 4 source tables, start with the 3 most important and stub the rest as pass-through staging models. Show me the model file list before running the compile.

Claude Code will ask about column structure before generating SQL. Answer based on your knowledge of the source data. If you have many tables, narrow to 3-4 before starting Phase 3 to keep this phase within the time target.

When Claude Code applies the content rules, review on the shared screen: no raw schema or table names in any UI string, fictional company name consistent, pipeline.json has a real fivetran.com `deep_link` .

Phase 3 done when: dbt compile and dbt ls pass, React shell generated, content rules confirmed.

Phase 4 — Run

target: 30 minutes

PHASE 4 — RUN

Before pasting the Phase 4 prompt — open a second terminal tab. Navigate to your demo frontend directory (`$HOME/Documents/GitHub/{demo_slug}-odi-demo/frontend/`) and run: `npm install && npm run dev` . The app will be available at `http://localhost:5173`. Keep this terminal running for the entire demo tour.

PASTE INTO CLAUDE CODE — BOTH OF YOU

This continues the Phase 1, 2, and 3 conversation. Recover the connector ID and bucket name from `infra/outputs.json` if needed.

Run Phase 4: Run.

In order:

1. Retrieve the connector ID from `infra/outputs.json` or from the `CONNECTOR_ID` value printed at the end of Phase 2. Trigger the first Fivetran sync: POST

`https://api.fivetran.com/v1/connectors/{connector_id}/force` using `FIVETRAN_API_KEY` and `FIVETRAN_API_SECRET`.

2. Write a file `sync_poll.sh` with this content and run it in a separate note — instruct the participant to run it in a second terminal tab:

```
while true; do
  STATUS=$(curl -s -u "${FIVETRAN_API_KEY}:${FIVETRAN_API_SECRET}"
https://api.fivetran.com/v1/connectors/{connector_id} | jq -r '.data.status.sync_state')
  echo "$(date): $STATUS"
  [[ "$STATUS" == "synced" || "$STATUS" == "failed" ]] && break
  sleep 30
done
```

Tell the participant: run `bash sync_poll.sh` in a second terminal tab and leave it running. This Claude Code session is available for review work while the sync runs.

3. If `sync_poll.sh` exits with "failed": print the Fivetran error and stop. Do not run dbt on failed data.

4. When `sync_poll.sh` exits with "synced": verify data landed in S3. For each source table, list `s3://{bucket}/{demo_slug}_bronze/{table_name}/metadata/` and confirm at least one `.json` file exists. Then run Athena `COUNT(*)` on each bronze table and print row counts.

5. Run: `dbt build --select "path:models/silver path:models/gold"` against the synced data. Report all results.

6. Walk through the demo tour with the React app running at `http://localhost:5173`:
- Landing page: org name visible, no raw table names, buyer persona copy correct

- Pipeline page: SQL Server connector card visible, Open-in-Fivetran link opens the correct connector at fivetran.com
- Fivetran dashboard at the connector deep link: status shows "synced"
- Athena: SELECT * LIMIT 5 on each gold table, confirm rows returned
- Analysis page at <http://localhost:5173/#/analysis>: axis labels visible, chart title shows fictional org name, live-data note visible

7. Print the build summary: demo slug, Fivetran group ID, connector ID and deep link, S3 bucket name, Glue database names, dbt project name, update method, sync status, count of gold models built.

8. Print what you built: a live Fivetran pipeline landing SQL Server CDC data into Apache Iceberg on S3, dbt-transformed bronze/silver/gold layers queryable via Athena, and a React frontend ready to demo the full ODI story to a CDO or CIO.

9. Print next steps: push repo to fivetran-jasonchletsos GitHub org, add to demo catalog, run demo-blitz for a full content and design pass, add analytical charts once data volume warrants.

While sync_poll.sh runs in the second terminal, use this time to review the dbt models and React frontend from Phase 3. Check: sources.yml table names match Phase 1, gold models use `table_type="iceberg"`, no hardcoded schema strings, company name renders in the React shell, connector card has a real fivetran.com URL. When sync completes, come back to Claude Code for the build and tour.

Phase 4 done when: sync confirmed, Athena row counts printed, dbt gold models populated, demo tour complete, summary and next steps printed.

When something breaks — how to handle it

The same approach works for every error in every phase:

1. Do not try to fix the error by editing files or rerunning commands yourself first.
2. Copy the full error output — the complete text, not just the last line.
3. Paste it into Claude Code and say: **"This failed. Here is the error. What is the fix?"**
4. Claude Code will diagnose it and give you the exact steps.

WHAT YOU SEE	WHAT TO PASTE TO CLAUDE CODE
<code>op read</code> returns nothing	Full terminal output + "op read returned empty — my vault name is X and item name is Y"
SQL Server port not reachable	Full nc or sqlcmd output + "connectivity check failed"
Terraform plan has unexpected resources	Full plan output + "I see resources I did not expect, please explain"
Fivetran API returns an error code	Full API response JSON + "the API returned an error"
dbt compile or build fails	Full dbt output including the error block + "dbt failed"
Athena query fails	Full error message + "Athena returned this error"
sync_poll.sh exits with "failed"	The sync_state value + the Fivetran connector status JSON + "sync failed"

If Claude Code cannot resolve the error within 15 minutes, the issue is likely outside your control — firewall blocking the SQL Server port, missing IAM permissions on a locked-down account, or CDC not enabled on the source database. Pause the session and escalate to whoever administers that system before continuing.

Hybrid deployment — the data-residency answer

Claude Code delivers this during Phase 2 as a short aside. It is also here for reference if a customer raises a data-residency concern during the demo.

Fivetran Hybrid Deployment runs the data-movement agent inside the customer's own network. Fivetran's cloud plane receives only orchestration metadata — row counts, schema, sync timestamps. No customer data transits Fivetran's infrastructure.

What stays identical to what you built today: the connector configuration, the MDLS destination, the dbt pipeline, the React frontend. Hybrid deployment is a deployment mode, not a different architecture.